

Image-Based Defect Detection for Manufacturing Inspection

In this project, you will use MATLAB to create a virtual inspection station that processes images of a single part type, produces defect evidence overlays and measurable features, and classifies parts using a pretrained network via transfer learning. You will then evaluate performance and robustness across a repeatable test suite.

Use this Live Script as a template to get started on your project solution. You may include helper functions at the bottom of this document or as separate files. For a tutorial on how to use Live Scripts, check out this [video](#).

Suggested Tasks

- 1. Explore and Organize the Image Dataset
- 2. Build a Single-Image Inspection Function
- 3. Train and Integrate an AI Classifier into your Single-Image Inspection Function
- 4. Evaluate Inspection System Performance
- 5. Test and Evaluate System Robustness

Table of Contents

Break Down the Problem.....	1
Task 1: Explore and Organize Image Dataset	1
Task 2: Build a Single-Image Inspection Function.....	2
Task 3: Train and Integrate an AI Classifier into your Single-Image Inspection Function.....	4
Task 3 Checkpoint: Combine outputs into a hybrid inspection result.....	4
Task 4: Evaluate Inspection System Performance.....	5
Task 5: Test and Evaluate System Robustness.....	6
Interpretation of Results.....	6

Break Down the Problem

In the space below, define the problem statement:

List the requirements, constraints, and success criteria for the project:

What is your proposed solution?

List the steps you will need to take to achieve your proposed solution:

What challenges do you face in the process of achieving the solution?

Task 1: Explore and Organize Image Dataset

Your goal in this step is to define defect classes and labels (keep it small).

- Load and inspect the images and labels from your chosen dataset (see the project page on GitHub for recommended datasets) to understand the part types, defect classes, and data balance
- Store labels in a table loaded by `readtable(...)`
- Manage images and labels with `imageDatastore(...)` and the `Labels` property (or use the example workflow for setting up labeled image datastores).
- Helpful reference: [Create and Explore Datastore for Image Classification](#).
- Choose 2-3 defect types (for FAIL) plus PASS

```
% Insert your code here (or make use of helper functions or additional .m  
or .mlx files and indicate where  
% they can be found). Ensure your code is well-documented.
```

Task 2: Build a Single-Image Inspection Function

Your goal in this step is to create a function that analyzes an image to identify and quantify suspected defects detected in the image. You're trying to answer the question: "What in the image looks suspicious?"

To build this function you will need to:

STEP 1. Standardize the images

- Read images with `imread(...)` or through `imageDatastore(...)`.
- Standardize size with `imresize(...)` so metrics and model input are consistent.
- Convert to grayscale with `rgb2gray(...)` as needed (e.g. classical processing may rely on intensity values)

STEP 2. Preprocess the images

- Apply classical preprocessing, prioritizing stability over complexity. In other words, apply only what you need to reduce sensitivity to normal station variation. Suggested processing includes:
- Lighting correction: `imflatfield(...)` (or background estimate with `imgaussfilt(...)` and subtraction)
- Contrast normalization: `adapthisteq(...)`
- Denoising: `medfilt2(...)` or `imgaussfilt(...)`
- Save intermediate results during development (e.g., `imshowpair(...)`) for quick debugging.

STEP 3. *Optionally*, define a region of interest (ROI)

If each image already contains a centered part with minimal clutter, skip ROI and use the full image:

- `roi = I;`

If needed, define ROI fully automatically (no per-image manual selection), by using, for example:

- Fixed crop defined once (camera is consistent): indexing or `imcrop(...)` OR
- Auto-crop from the part mask: segment the part, compute BoundingBox using `regionprops(...)`, then `imcrop(...)`.

STEP 4. Segment the image for an evidence overlay

Your goal is to generate an evidence overlay and a few measurable numbers—not a perfect segmentation. Generate a simple binary mask that highlights suspicious regions in the image. Use a defect-appropriate recipe and keep it short:

- Start with thresholding: `imbinarize(...)` or `adaptthresh(...)` + `imbinarize(...)`
- Clean small specks: `bwareaopen(...)`
- Bridge gaps (if appropriate): `imclose(...)`
- Fill holes (only when it matches your goal (e.g., for a clean outer part silhouette): `imfill(..., 'holes')`

Deliverable: `maskEvidence = defectEvidence(roi)`

STEP 5. Extract a small set of interpretable metrics

Your goal is to quantify the evidence for suspected defects, which supports traceability and debugging in your workflow. These metrics should answer the question: “What did the algorithm see, and how strong was the evidence?”

These measurements are mainly for traceability (inspection logs), debugging, and sanity checks. Keep it to 3–4 metrics so it stays easy.

Recommended minimal metrics (works for many defect types):

- `numComponents`: how many separate suspicious regions were found (use `bwconncomp(maskEvidence)` and `readNumObjects`)
- `maxArea`: size of the largest suspicious region (use `regionprops(maskEvidence, 'Area')`)
- `areaRatio`: fraction of the ROI flagged as suspicious (`nnz(maskEvidence) / numel(maskEvidence)`)
- (*optional*) `edgeDensity`: if your evidence comes from edges (chips/scratches), measure how “edgy” the ROI is (`nnz(edgeMask) / numel(edgeMask)`)

STEP 6. *Optionally*, implement a rule-based baseline.

Implement 2–3 conservative rules so your system can always produce an explainable fallback decision mechanism. This can also be a useful comparison with AI result.

- Example: fail if `maxArea > Amax` or `numComponents > Nmax`

Deliverable: `baselineDecision = decideRules(evidence)`

```
% Insert your code here (or make use of helper functions or additional .m
or .mlx files and indicate where
% they can be found). Ensure your code is well-documented.
```

Task 3: Train and Integrate an AI Classifier into your Single-Image Inspection Function

Your goal in this step is to use a MathWorks provided pretrained network (recommended: resnet18) to classify each part as either a PASS or FAIL, that will be part of the output of the function you began to build in the previous step.

- Load the recommended starter network: use `imagePretrainedNetwork` and select `resnet18` from the Deep Learning Toolbox.
- Resize/augment input with `augmentedImageDatastore(...)` to match the network input size.
- Fine-tune the last layers for your classes (start with PASS vs FAIL), using `trainNetwork(...)` or a guided workflow such as [Get started with transfer learning](#) or [pretrained network workflows](#).
- The output of your AI classification model should be a predicted label (PASS/FAIL) and a confidence score in that label

Deliverable: `[aiLabel, aiScore] = classify(net, roiForNet)`

```
% Insert your code here (or make use of helper functions or additional .m
or .mlx files and indicate where
% they can be found). Ensure your code is well-documented.
```

```
function [aiLabel, aiScores] = classify(net, roiForNet)
    % You can write your function here
end
```

Task 3 Checkpoint: Combine outputs into a hybrid inspection result

Your image inspection function (e.g. `inspectPar(I)`) should combine the classical evidence with the AI classification output.

In other words, your image inspection function should integrate both classical evidence from image processing and an AI classification decision. The AI classifier provides the main decision (PASS/FAIL) while the classical evidence extraction provides interpretability, traceability, and sanity checks for that decision.

Your function should return:

- finalLabel (from AI, in Task 3)
- confidenceScore (from AI, in Task 3)
- evidenceOverlay (from Step 4 in Task 2)
- evidenceMetrics (from Step 5 in Task 2)
- (optional) baselineDecision and a “disagreement flag” if optional rules in Step 6 from Task 2 and AI output disagree

Optionally, you can also choose to overlay these results on the image using `insertShape(...)` / `insertText(...)`, and save rejects with `imwrite(...)`

```
% Insert your code here (or make use of helper functions or additional .m
or .mlx files and indicate where
% they can be found). Ensure your code is well-documented.
```

```
function [finalLabel, confidenceScore, evidenceOverlay, evidenceMetrics,
baselineDecision] = inspectPart(I)
    % You can write your function here
end
```

Task 4: Evaluate Inspection System Performance

Now that you've built a hybrid inspection system that integrates classical evidence with an AI decision, your goal in this step is to test the system on a batch of images.

The results of this test should be summarized with a confusion matrix, report yield, and defect rates across the entire batch of images.

Write a script (e.g. `runInspectionSuite.m`) that:

- Splits data into train/test (e.g., `splitEachLabel(...)` or a fixed split)
- Evaluates the trained AI model and trains the inspection system on a test set and logs outcomes
- Produces a confusion matrix with `confusionmat(...)` / `confusionchart(...)`
- Summarizes yield and defect counts in tables and plots
- Visualizes common failure cases (e.g. by creating a montage of common errors using `montage(...)`)

```
% Here, it is recommended to create a separate runInspectionSuite.m file
% (or include as a local helper function) that combines the work you've
done
% so far here and additionally performs the steps outlined in the task
description.
% You can run it from here.
```

Task 5: Test and Evaluate System Robustness

Your goal in this step is to assess how your system performs under simulated variations. You can simulate image inspection station variation by altering the lighting, blur, or noise in the images. Here are some suggested distortions to apply:

- Brightness/contrast: `imadjust(...)`
- Blur: `imgaussfilt(...)`
- Noise: `imnoise(...)`

Then, re-run the evaluation. In other words, run `runInspectionSuite.m` again with the same image set after applying distortions to the images.

Compare how performance changes across conditions by reporting how accuracy and false-reject rates change under simulated variations.

```
% Insert your code here (or make use of helper functions or additional .m  
or .mlx files and indicate where  
% they can be found). Ensure your code is well-documented.
```

Interpretation of Results

Summarize your findings by interpreting their physical/engineering meaning:

What are some limitations of your work?

What are practical next steps?